

当 `vector v` 被销毁, 它有责任销毁其内含的所有 `widgets`。假设 `v` 内含十个 `widgets`, 而在析构第一个元素期间, 有个异常被抛出。其他九个 `widgets` 还是应该被销毁 (否则它们保存的任何资源都会发生泄漏), 因此 `v` 应该调用它们各个析构函数。但假设在那些调用期间, 第二个 `widget` 析构函数又抛出异常。现在有两个同时作用的异常, 这对 C++ 而言太多了。在两个异常同时存在的情况下, 程序若不是结束执行就是导致不明确行为。本例中它会导致不明确的行为。使用标准程序库的任何其他容器 (如 `list`, `set`) 或 TR1 的任何容器 (见条款 54) 或甚至 `array`, 也会出现相同情况。容器或 `array` 并非遇上麻烦的必要条件, 只要析构函数吐出异常, 即使并非使用容器或 `arrays`, 程序也可能过早结束或出现不明确行为。是的, C++ 不喜欢析构函数吐出异常!

这很容易理解, 但如果你的析构函数必须执行一个动作, 而该动作可能会在失败时抛出异常, 该怎么办? 举个例子, 假设你使用一个 `class` 负责数据库连接:

```
class DBConnection {
public:
    ...
    static DBConnection create();           //这个函数返回
                                           // DBConnection 对象;
                                           //为求简化暂略参数。
    void close();                           //关闭联机; 失败则抛出异常。
};
```

为确保客户不忘记在 `DBConnection` 对象身上调用 `close()`, 一个合理的想法是创建一个用来管理 `DBConnection` 资源的 `class`, 并在其析构函数中调用 `close`。这一类用于资源管理的 `classes` 在第 3 章有详细探讨, 这儿只要考虑它们的析构函数长相就够了:

```
class DBConn {                               //这个 class 用来管理 DBConnection 对象
public:
    ...
    ~DBConn()                               //确保数据库连接总是会被关闭
    {
        db.close();
    }
private:
    DBConnection db;
};
```

这便允许客户写出这样的代码:

```

{
    DBConn dbc(DBConnection::create());
    ...
}
//开启一个区块 (block)。
//建立 DBConnection 对象并
// 交给 DBConn 对象以便管理。
//通过 DBConn 的接口
// 使用 DBConnection 对象。
//在区块结束点, DBConn 对象
// 被销毁, 因而自动
// 为 DBConnection 对象调用 close

```

只要调用 close 成功, 一切都美好。但如果该调用导致异常, DBConn 析构函数会传播该异常, 也就是允许它离开这个析构函数。那会造成问题, 因为那就是抛出了难以驾驭的麻烦。

两个办法可以避免这一问题。DBConn 的析构函数可以:

- 如果 close 抛出异常就结束程序。通常通过调用 abort 完成:

```

DBConn::~DBConn( )
{
    try { db.close(); }
    catch (...) {
        制作运转记录, 记下对 close 的调用失败;
        std::abort();
    }
}

```

如果程序遭遇一个“于析构期间发生的错误”后无法继续执行, “强迫结束程序”是个合理选项。毕竟它可以阻止异常从析构函数传播出去(那会导致不明确的行为)。也就是说调用 abort 可以抢先制“不明确行为”于死地。

- 吞下因调用 close 而发生的异常:

```

DBConn::~DBConn( )
{
    try { db.close(); }
    catch (...) {
        制作运转记录, 记下对 close 的调用失败;
    }
}

```

一般而言, 将异常吞掉是个坏主意, 因为它压制了“某些动作失败”的重要信息! 然而有时候吞下异常也比负担“草率结束程序”或“不明确行为带来的风险”好。为了让这成为一个可行方案, 程序必须能够继续可靠地执行, 即使在遭遇并忽略一个错误之后。

这些办法都没什么吸引力。问题在于两者都无法对“导致 close 抛出异常”的情况做出反应。

一个较佳策略是重新设计 DBConn 接口, 使其客户有机会对可能出现的问题作出反应。例如 DBConn 自己可以提供一个 close 函数, 因而赋予客户一个机会得以处理“因该操作而发生的异常”。DBConn 也可以追踪其所管理之 DBConnection 是否已被关闭, 并在答案为否的情况下由其析构函数关闭之。这可防止遗失数据库连接。然而如果 DBConnection 析构函数调用 close 失败, 我们又将退回“强迫结束程序”或“吞下异常”的老路:

```
class DBConn {
public:
    ...
    void close( )                //供客户使用的新函数
    {
        db.close( );
        closed = true;
    }
    ~DBConn()
    {
        if (!closed) {
            try {                //关闭连接(如果客户不那么做的话)
                db.close( );
            }
            catch (...) {        //如果关闭动作失败,
                                // 记录下来并结束程序
                                // 或吞下异常。
                ...
            }
        }
    }
private:
    DBConnection db;
    bool closed;
};
```

把调用 close 的责任从 DBConn 析构函数手上移到 DBConn 客户手上(但 DBConn 析构函数仍内含一个“双保险”调用)可能会给你“肆无忌惮转移负担”的印象。你甚至可能认为它违反条款 18 所提忠告(让接口容易被正确使用)。实际上这两项污名都不成立。如果某个操作可能在失败时抛出异常, 而又存在某种需要必须处理该异常, 那么这个异常必须来自析构函数以外的某个函数。因为析构函数吐出异常

就是危险，总会带来“过早结束程序”或“发生不明确行为”的风险。本例要说的是，由客户自己调用 `close` 并不会对他们带来负担，而是给他们一个处理错误的机会，否则他们没机会响应。如果他们不认为这个机会有用（或许他们坚信不会有错误发生），可以忽略它，倚赖 `DBConn` 析构函数去调用 `close`。如果真有错误发生——如果 `close` 的确抛出异常——而且 `DBConn` 吞下该异常或结束程序，客户没有立场抱怨，毕竟他们曾有机会第一手处理问题，而他们选择了放弃。

请记住

- 析构函数绝对不要吐出异常。如果一个被析构函数调用的函数可能抛出异常，析构函数应该捕捉任何异常，然后吞下它们（不传播）或结束程序。
- 如果客户需要对某个操作函数运行期间抛出的异常做出反应，那么 `class` 应该提供一个普通函数（而非在析构函数中）执行该操作。

条款 09：绝不在构造和析构过程中调用 virtual 函数

Never call virtual functions during construction or destruction.

本条款开始前我要先阐述重点：你不该在构造函数和析构函数期间调用 `virtual` 函数，因为这样的调用不会带来你预想的结果，就算有你也不会高兴。如果你同时也是一位 Java 或 C# 程序员，请更加注意本条款，因为这是 C++ 与它们不相同的一个地方。

假设你有个 `class` 继承体系，用来塑模股市交易如买进、卖出的订单等等。这样的交易一定要经过审计，所以每当创建一个交易对象，在审计日志（audit log）中也需要创建一笔适当记录。下面是一个看起来颇为合理的做法：

```
class Transaction {                                //所有交易的 base class
public:
    Transaction( );
    virtual void logTransaction() const = 0;        //做出一份因类型不同而不同
                                                    //的日志记录 (log entry)
    ...
};
```